

Technical Root Cause Analysis

Subject: Missing Server-Side Python Lifecycle Hooks in ERPNext HCP Profiling Approval Workflow

Target System: ERPNext v14/v15 Dev Server (`dev.pmi-marketing.com`)

Date of Audit: September 10, 2026 | **Status:** Definite Findings Confirmed by Database Query

Executive Summary of the Architectural Gap:

The ERPNext server-side is completely missing the Python document lifecycle hook (`on_submit()` or `doc_events` in `hooks.py`) on the HCP Profile Submission DocType controller.

In Frappe/ERPNext, native workflows (Workflow & Workflow Action) only change the document's own state from Pending Approval to Approved and flip `docstatus = 1` . **Workflows do not spawn or update records in other DocTypes (HCP , HCP Account) without custom Python backend code.**

1. Concrete Database Evidence on `dev.pmi-marketing.com`

A direct audit of all 27 `Approved` submissions on the live ERPNext server revealed an exact pattern:

Submission ID	Doctor Full Name	Approval Channel	hcp_name (Linked ID)	Master HCP Record	HCP Account Record	Resulting Status
HCP-PROF-2026-00153	Yannie Hanna Joy	FLUTTER APP	HCP-0000043	CREATED	SYNCED	Fully Operational
HCP-PROF-2026-00150	Leandro Miguel Santos	FLUTTER APP	HCP-0000042	CREATED	SYNCED	Fully Operational
HCP-PROF-2026-00124	Maria Sol Bernades	ERPNext DESK	NULL / ""	MISSING	MISSING	Broken / Orphaned State
HCP-PROF-2026-00118	8 8 8	ERPNext DESK	NULL / ""	MISSING	MISSING	Broken / Orphaned State
HCP-PROF-2026-00110	2 2 2	ERPNext DESK	NULL / ""	MISSING	MISSING	Broken / Orphaned State

When an administrator navigates to ERPNext Desk in a web browser and clicks the workflow button "**Approve**", Frappe seals the document with `docstatus = 1` , but leaves `hcp_name` blank, generates **zero** records in `HCP` , and generates **zero** records in `HCP Account` .

2. The 4 Specific Root Causes on the ERPNext Server

Cause A: Only Browser-Side Client Scripts Exist (No Backend Code)

Inspection of `dev.pmi-marketing.com/api/resource/Client` Script shows the developer (`/dah`) wrote 38,045 bytes of JavaScript in `HCP Profile Submission-Client` up to version `v0.1.6` . This script only handles:

- Calculating the full name string concatenation in the browser form.

- Tracking JSON change summaries for UI review.
- Enforcing checkbox toggles for "preferred" clinics and specialties.

Key Fact: Client Scripts only run in a human user's web browser when looking at the Desk form. They do not run during automated workflow state transitions or backend submissions.

Cause B: Frappe Server Scripts are Blocked (HTTP 403 FORBIDDEN)

When querying `/api/resource/Server Script`, the server returned `403 Forbidden`. In production Frappe sites, `server_script_enabled` is set to `false` in `common_site_config.json` for security. Therefore, Python logic **cannot** be added via the ERPNext web interface; it must be written in the app's backend repository on the server file system.

Cause C: Missing `on_submit()` in `hcp_profile_submission.py`

In Frappe apps, when a document transitions through a workflow to `docstatus = 1`, Frappe automatically triggers `on_submit()` on the DocType's Python controller class. Currently, the server controller contains no logic to instantiate or update foreign DocTypes.

Cause D: Frappe Workflow Engine Does Not Perform Cross-DocType Mutations

The Workflow Engine (`frappe.model.workflow.apply_workflow`) is strictly a state machine for the current document table. It updates `workflow_state = 'Approved'` and `docstatus = 1`. It does not know that an `HCP Profile Submission` is supposed to spawn an `HCP` and sync an `HCP Account`.

3. Architecture Comparison: ERPNext Desk vs. Mobile App Saga

(Current Broken Behavior): - Manager Clicks [Approve] ▼ <code>frappe.model.workflow.apply_workflow</code> ▼ <code>workflow_state = 'Approved', docstatus = 1 (SEALED)</code> <code>hcp_name = NULL</code> ❌ NO Master Doctor created in HCP doctype ❌ NO Program Record created in HCP Account doctype	1. ERPNext Desk Web Flow
(Working Client-Side Saga Orchestrator): - Manager Clicks [Approve & Sync Masterlist] → Phase 1: POST <code>/api/resource/HCP</code> → Creates Universal Doctor (Returns HCP-0000043) → Phase 2: POST <code>/api/resource/HCP Account</code> → Provisions Account with Territory & Manager → Phase 3: POST <code>apply_workflow / set_value</code> → Sets <code>workflow_state = 'Approved', docstatus = 1</code>, and binds <code>hcp_name = 'HCP-0000043'</code>	2. HCP Mobile App Flow

4. The Required ERPNext Server-Side Python Fix

To fix the approval process on ERPNext Desk, the backend developer must add the following implementation to `hcp_profile_submission.py` (or via a `doc_events` hook in `hooks.py`):

```
import frappe
from frappe.model.document import Document

class HCPProfileSubmission(Document):
    def on_submit(self):
        # Executed automatically when workflow reaches Approved (docstatus = 1)
        if self.workflow_state == "Approved":
```

```

effective_hcp_id = self.hcp_name

# 1. Provision Universal Doctor Master (HCP DocType)
if self.profile_action == "New HCP" or not effective_hcp_id:
    new_hcp = frappe.get_doc({
        "doctype": "HCP",
        "first_name": self.first_name,
        "middle_name": self.middle_name or "-",
        "last_name": self.last_name,
        "hcp_full_name": self.hcp_full_name,
        "birth_date": self.birth_date,
        "hcp_type": self.hcp_type,
        "hcp_practice": self.hcp_practice,
        "hcp_specialty": [{"hcp_specialty": s.hcp_specialty, "sub_specialty": s.sub_specialty} for s in self.hcp_specialties],
        "hcp_workplace": [{"hcp_workplace": w.hcp_workplace, "address": w.workplace_name} for w in self.hcp_workplaces],
        "hcp_contact_info": [{"contact_number": c.contact_number, "email_address": c.email_address} for c in self.hcp_contact_info]
    })
    new_hcp.insert(ignore_permissions=True)
    effective_hcp_id = new_hcp.name

# Link newly created doctor back to this submission
self.db_set("hcp_name", effective_hcp_id)

# 2. Provision or Additively Update Program Master (HCP Account DocType)
acc_name = frappe.db.get_value(
    "HCP Account",
    {"hcp": effective_hcp_id, "account_or_program": self.account_or_program},
    "name"
)

if not acc_name:
    new_acc = frappe.get_doc({
        "doctype": "HCP Account",
        "hcp": effective_hcp_id,
        "hcp_name": self.hcp_full_name,
        "account_or_program": self.account_or_program,
        "territory": self.territory,
        "sales_person": self.sales_person,
        "specialization": [{"hcp_specialty": s.hcp_specialty, "preferred": s.preferred} for s in self.hcp_specialties],
        "workplace_info": [{"hcp_workplace": w.hcp_workplace, "preferred": w.preferred} for w in self.hcp_workplaces],
        "contact_info": [{"contact_number": c.contact_number, "preferred": c.preferred} for c in self.hcp_contact_info]
    })
    new_acc.insert(ignore_permissions=True)
else:
    acc_doc = frappe.get_doc("HCP Account", acc_name)
    # Perform additive merge on child tables...
    acc_doc.save(ignore_permissions=True)

```

Conclusion:

Until the ERPNext backend developer deploys this Python controller hook to the server, **approvals performed inside ERPNext Desk will continue to leave submissions in a broken, half-committed state** with empty `hcp_name` and missing doctors. The Flutter mobile app remains the **only** client capable of completing full multi-doctype masterlist provisioning.